NLP (Natural Language Processing)

What is NLP?

"NLP stands for Natural Language Processing. It is a branch of Artificial Intelligence that helps computers understand, process, and generate human language.

The main goal of NLP is to make communication between humans and computers easier using natural language like English, Telugu, or Hindi.

NLP is used for tasks such as sentiment analysis, text classification, machine translation, chatbots, spam detection, speech recognition, and question answering.

For example, when we type a question into ChatGPT, NLP techniques help the system understand the meaning of our question and generate a relevant response."

Key Points to Memorize

Remember these **4 words**:

NLP = Understand → Process → Analyze → Generate

- 🤖 Branch of **AI**
- 📝 Works with **human language**
- □ Helps computers understand language
- 💬 Used in chatbots, translation, sentiment analysis, etc.

🔹 **Small Example**

Suppose a customer says:

"The product is amazing!"

NLP can identify that the sentence has a **positive sentiment**.

💡 **Easy Memory Trick**

NLP = Human Language → Computer Understanding

Text Preprocessing in NLP

"Text preprocessing is the process of cleaning and preparing raw text before giving it to a machine learning or NLP model.

Raw text usually contains unnecessary information like punctuation, special characters, extra spaces, and different forms of the same word. Preprocessing converts this raw text into a clean and useful format.

Common preprocessing steps include **lowercasing, tokenization, removing unnecessary stop words, removing punctuation, stemming or lemmatization, and handling special characters**.

For example, if the input is 'I am LOVING this product!!!', we can convert it into a cleaner form such as 'loving product'. This makes the text easier for the NLP model to process."

□ Key Points to Memorize

Remember:

Clean → Split → Remove → Normalize

1. **Lowercasing** → Hello → hello
2. **Tokenization** → sentence → words/tokens
3. **Stop-word removal** → remove unnecessary words like is, the, a
4. **Punctuation removal** → Hello!!! → Hello
5. **Stemming** → playing → play
6. **Lemmatization** → better → good *(based on context)*

◈ Small Example

Input:

"The cats are playing!!!"

After preprocessing:

cats playing

Then tokenization:

["cats", "playing"]

⚠ Important Interview Point

Don't say **"we always remove stop words and punctuation."**

That's not always true.

For example, in **sentiment analysis**, words like **"not"** can be very important:

"This movie is **not** good."

Removing not can completely change the meaning.

Tokenization

"Tokenization is the process of breaking text into smaller units called tokens. These tokens can be words, subwords, or even characters, depending on the tokenizer.

It is one of the important steps in NLP because machine learning models cannot directly understand raw text. Tokenization converts the text into manageable pieces that can later be converted into numerical representations.

For example, the sentence ‘I love NLP’ can be tokenized into three word tokens: I, love, and NLP.

Modern models like BERT and GPT commonly use **subword tokenization**, which helps handle unknown or rare words more effectively.”

□ Key Points to Memorize

Tokenization = Breaking text → Tokens

Types:

1. **Word tokenization** → I love NLP → I | love | NLP
2. **Sentence tokenization** → paragraph → sentences
3. **Character tokenization** → cat → c | a | t
4. **Subword tokenization** → word → smaller meaningful pieces

◈ Small Example

Sentence:

“I am learning NLP.”

Word tokens:

["I", "am", "learning", "NLP"]

⚠ Important Interview Point

Token ≠ Always a word.

A token can be a **word, subword, or character** depending on the tokenizer.

For example, a rare word might be split into multiple subword tokens.

Stop Words

“Stop words are common words that usually carry less important information for a particular NLP task. Examples include words like ‘the’, ‘is’, ‘a’, ‘an’, ‘and’, and ‘of’.

In traditional NLP preprocessing, we may remove these words to reduce the size of the text and focus on more meaningful words.

For example, in the sentence ‘The cat is sitting on the table’, we might remove words like ‘the’, ‘is’, and ‘on’, leaving ‘cat sitting table’.

However, stop-word removal is not always necessary. It depends on the task. For example, in sentiment analysis, removing the word **'not'** could change the meaning completely. So we should remove stop words only when it is useful for the specific NLP task."

□ Key Points to Memorize

Stop Words = Common words with less useful information

Examples:

the, is, a, an, and, of, on

Remember:

"Don't blindly remove stop words."

It depends on the **NLP task**.

🔷 Small Example

Original:

"This movie is not good."

If we remove not:

"movie good" ❌

The meaning changes!

So for **sentiment analysis**, not is important.

⚠ Interview Confusion

Are stop words always removed?

👉 No.

Modern transformer/LLM-based models generally **do not use traditional stop-word removal as a mandatory preprocessing step**. They tokenize the full input and learn which words are important from context.

Stemming vs Lemmatization

"Stemming and lemmatization are techniques used to reduce words to their basic form. The main difference is that stemming simply cuts or removes parts of a word, while lemmatization uses vocabulary and language rules to find the meaningful base word.

For example, stemming may convert 'playing' to 'play', but sometimes it can produce an incorrect word. Lemmatization generally produces a valid dictionary word.

Stemming is usually faster because it uses simple rules, while lemmatization is more accurate but can take more processing time.

So, if speed is more important, we can use stemming. If we need linguistically meaningful words, lemmatization is usually preferred."

□ Key Points to Memorize

STEMMING = Cutting ✂

LEMMATIZATION = Understanding 🕮

	Stemming	Lemmatization
Method	Cuts word	Uses language rules
Speed	Faster ⚡	Slower
Accuracy	Lower	Higher
Output	May not be a real word	Usually valid word

◈ Small Example

Words:

playing, played, plays

Stemming:

→ play *(or sometimes an imperfect form depending on the stemmer)*

Lemmatization:

→ play

Another example:

"better"

Lemmatization can understand its base form as:

→ "good"

□ Easy Memory Trick

Stem = Cut the word ✂
Lemma = Understand the word □

⚠ Interview Follow-up

Interviewer: "Which one would you choose for a real-world NLP project?"

Good answer:

"It depends on the application. If I need fast and simple preprocessing, I may use stemming. If semantic accuracy and meaningful dictionary forms are important, I would prefer lemmatization."

Bag of Words (BoW)

This is a **very important basic NLP interview topic** because it introduces how we convert text into numbers. 🔥

🎯 Interview Answer — 1–2 minutes

"Bag of Words, or BoW, is a simple technique used to represent text as numerical vectors based on the words present in the text.

First, we create a vocabulary containing all unique words from the documents. Then, for each sentence or document, we count how many times each vocabulary word appears.

The important point is that BoW considers word occurrence or frequency, but it does not understand the meaning or order of words.

For example, if we have 'I like Python' and 'I like NLP', the vocabulary is 'I, like, Python, NLP'. Each sentence can then be represented as a vector of word counts.

BoW is simple and useful for tasks like text classification, but it can create large and sparse vectors and cannot capture the semantic meaning of words."

□ Key Points to Memorize

BoW = Text → Vocabulary → Word Counts → Numbers

Remember these **3 limitations**:

- ❌ Doesn't understand word meaning
- ❌ Doesn't consider word order
- ❌ Can create large **sparse vectors**

🔷 Small Example

Two sentences:

"I like Python"
"I like NLP"

Vocabulary:

[I, like, Python, NLP]

Vectors:

I like Python → [1, 1, 1, 0]

I like NLP → [1, 1, 0, 1]

Here, 1 means the word is present and 0 means it is absent.

⚠️ Important Interview Question

Interviewer: “Does Bag of Words understand the meaning of words?”

👉 **No.**

For example:

“Dog bites man”
“Man bites dog”

BoW sees the same words with similar counts, even though the meanings are completely different because the **word order changes**.

□ One-Line Memory

“BoW counts words, but doesn't understand them.”

TF-IDF

TF-IDF = Term Frequency – Inverse Document Frequency

This is one of the **most important traditional NLP interview questions.**

🎯 **Interview Answer — 1–2 minutes**

“TF-IDF is a technique used to convert text into numerical values by measuring how important a word is in a document compared to a collection of documents.

TF means Term Frequency, which tells us how frequently a word appears in a particular document. IDF means Inverse Document Frequency, which reduces the importance of words that appear in many documents.

So, a word gets a high TF-IDF score when it appears frequently in one document but is relatively rare across the entire collection.

For example, words like ‘the’ may appear in almost every document, so they get a low score. But a specific word like ‘neural’ may appear frequently in one document and rarely in others, so it can get a higher score.

TF-IDF is commonly used for text classification, document search, and information retrieval.”

□ Key Points to Memorize

Remember:

TF-IDF = Local importance × Global rarity

- **TF** → How often the word appears in **this document**
- **IDF** → How rare the word is across **all documents**
- Common words → **Low score**

- Important/rare words → **High score**

🔷 Small Example

Suppose we have 100 documents.

"the" appears in 95 documents → very common → **low IDF**

"transformer" appears in 5 documents → rare → **high IDF**

If **"transformer"** also appears many times in one particular document:

👉 High TF-IDF score ✅

🧠 Easy Memory Trick

TF = Frequent here 📄
IDF = Rare everywhere 🌍

So:

TF-IDF = Frequent here + Rare elsewhere = Important word

⚠️ BoW vs TF-IDF

BoW:

"How many times does this word appear?"

TF-IDF:

"How important is this word compared with other documents?"

Word Embeddings

"understanding relationships between words."

"Word embedding is a technique used to represent words as numerical vectors in a continuous vector space. Unlike Bag of Words or TF-IDF, embeddings can capture relationships and similarities between words based on their context.

The basic idea is that words with similar meanings tend to have similar vector representations. For example, 'king' and 'queen' will have vectors that are closer to each other than 'king' and 'banana'.

Word embeddings can be generated using techniques such as Word2Vec, GloVe, and FastText. Modern NLP models like BERT also create contextual embeddings, where the representation of a word can change depending on the sentence.

These embeddings are useful for tasks such as sentiment analysis, text classification, semantic search, and question answering."

🧠 Key Points to Memorize

Word Embedding = Word → Numerical Vector

Remember:

- 📊 Represents words as **vectors**
- □ Captures **semantic relationships**
- 🔗 Similar words → similar vectors
- 📚 Word2Vec, GloVe, FastText
- 🤘 BERT → **contextual embeddings**

🔷 Small Example

Imagine:

King → [0.8, 0.2, 0.9]
Queen → [0.7, 0.3, 0.8]
Banana → [0.1, 0.9, 0.2]

King and Queen have similar representations because they are semantically related.

⚠️ Very Important Interview Difference

TF-IDF:

Represents a word based mainly on its **importance/frequency in documents**.

Word Embedding:

Represents a word based on its **meaning and relationships with other words**.

□ Easy Memory Trick

BoW → Counts words
TF-IDF → Finds important words
Embeddings → Captures word relationships

Word2Vec

"Word2Vec is a neural-network-based technique used to create word embeddings. It represents words as vectors in such a way that words appearing in similar contexts have similar vector representations.

Word2Vec mainly has two architectures: **CBOW and Skip-gram**.

In **CBOW**, the model uses surrounding words to predict the target word. For example, in 'I drink ___ every morning', the surrounding words can help predict 'coffee'.

In **Skip-gram**, the model does the opposite: it uses a target word to predict the surrounding words.

After training on a large amount of text, Word2Vec produces vectors that capture useful relationships between words. These vectors can then be used in tasks such as text classification, sentiment analysis, and similarity measurement."

□ Key Points to Memorize

Word2Vec = Words → Meaningful Vectors

Two architectures:

Architecture	What does it predict?
CBOW	Context → Target word
Skip-gram	Target word → Context words

🔷 Small Example

Sentence:

"I drink coffee every morning."

CBOW:

I + drink + every + morning → coffee

👉 Predict the **middle/target word**.

Skip-gram:

coffee → drink, every, morning

👉 Predict the **surrounding words**.

□ Easy Memory Trick

CBOW = Context → Word
Skip-gram = Word → Context

That's the **most important thing to remember**. 🔥

⚠ Interview Follow-up

Interviewer: "Which is better, CBOW or Skip-gram?"

Good answer:

"It depends on the data and task. CBOW is generally faster and works well with frequent words, while Skip-gram is often better at learning representations for rare words."

💡 One Important Limitation

Traditional Word2Vec gives **one vector for a word**, regardless of its context.

Example:

“I went to the **bank** to deposit money.”

“I sat near the **bank** of the river.”

Traditional Word2Vec doesn't create completely different representations for these two meanings.

CBOW vs Skip-gram

“CBOW and Skip-gram are the two main architectures of Word2Vec. The main difference is the direction of prediction.

CBOW, or Continuous Bag of Words, uses the surrounding context words to predict the target word. For example, if the sentence is ‘I drink coffee every morning’, CBOW can use words like ‘I’, ‘drink’, ‘every’, and ‘morning’ to predict ‘coffee’.

Skip-gram does the opposite. It takes the target word and predicts its surrounding context words. So, given ‘coffee’, it tries to predict words such as ‘drink’, ‘every’, and ‘morning’.

CBOW is generally faster and works well with frequent words, while Skip-gram can perform better for rare words and smaller datasets.”

□ Key Points to Memorize

CBOW:
👉 Context → Target

Skip-gram:
👉 Target → Context

🔷 Small Example

Sentence:

“I love machine learning.”

CBOW:

I + love + learning → machine

Skip-gram:

machine → I, love, learning

Quick Comparison

	CBOW	Skip-gram
Input	Context words	Target word
Predicts	Target word	Context words

	CBOW	Skip-gram
Speed	Faster ⚡	Slower
Rare words	Usually weaker	Usually better
Training	More efficient	More computationally expensive

Memory Trick

CBOW = Context Before/around → Word
Skip-gram = Skip from Word → surrounding words

Or simply remember:

CBOW: Many → One
Skip-gram: One → Many 🔥

💬 **Interview Follow-up**

Interviewer: *"What is the major limitation of Word2Vec?"*

Answer:

"Word2Vec generates a fixed representation for each word, so it cannot properly handle words with different meanings in different contexts."

Example:

"I deposited money in the bank."
"The river bank is beautiful."

The word **bank** has different meanings, but traditional Word2Vec gives it essentially the same embedding.

GloVe

GloVe = Global Vectors for Word Representation

"GloVe is a word embedding technique used to represent words as numerical vectors. It learns word representations by using the statistical information of how frequently words appear together across a large collection of text.

The main idea is that the meaning of a word can be understood from the words that commonly appear around it.

Unlike Word2Vec, which mainly learns from local context windows, GloVe combines **global word co-occurrence statistics** with vector representations.

For example, words like 'king', 'queen', 'man', and 'woman' appear in related contexts, so GloVe can learn meaningful relationships between their vectors.

GloVe embeddings can be used for tasks such as text classification, sentiment analysis, and semantic similarity.”

□ Key Points to Memorize

GloVe = Global word co-occurrence → Word vectors

Remember:

- 📊 Uses **word co-occurrence statistics**
- 🌍 Captures **global information**
- 🔢 Converts words into vectors
- □ Similar contexts → similar representations

🔷 Small Example

Suppose a large collection contains:

coffee → drink, cup, hot, morning

tea → drink, cup, hot, morning

Because **coffee** and **tea** frequently occur with similar words, GloVe learns similar vector representations.

⚡ GloVe vs Word2Vec

This is the **most important interview comparison**:

Word2Vec	GloVe
Focuses on predicting words from context	Uses word co-occurrence statistics
More **local-context** based	Uses **global** corpus information
Neural prediction approach	Matrix/co-occurrence-based approach
CBOW & Skip-gram	No CBOW/Skip-gram architecture

□ Easy Memory Trick

Word2Vec → Predict 🎯
GloVe → Co-occurrence 📊

⚠ Important Limitation

Like traditional Word2Vec, **GloVe produces one fixed vector for a word**.

So:

“I went to the **bank**.”

and

“The river **bank** is beautiful.”

The word **bank** doesn't get a truly different contextual representation.

This limitation is one reason **contextual embeddings like BERT** became important.

Static vs Contextual Embeddings

This is an **important bridge from Word2Vec/GloVe → BERT/GPT**. 🔥

“The main difference between static and contextual embeddings is how they represent a word when its meaning changes based on the sentence.

Static embeddings give a single fixed vector to each word. Techniques like Word2Vec and GloVe are examples. So, the word ‘bank’ gets essentially the same vector whether we talk about a financial bank or a river bank.

Contextual embeddings generate a representation based on the surrounding words. Models such as BERT create contextual representations, so the representation of ‘bank’ can change depending on the sentence.

Therefore, static embeddings are simpler and capture general word relationships, while contextual embeddings are better at understanding the meaning of words in their specific context.”

□ Key Points to Memorize

Static = One word → One vector

Contextual = One word → Different vector depending on context

Examples:

- **Word2Vec** → Static
- **GloVe** → Static
- **BERT** → Contextual

🔷 Small Example

Sentence 1:

“I deposited money in the **bank**.”

Bank = financial institution 🏢

Sentence 2:

“We sat near the river **bank**.”

Bank = side of a river 🌊

Static Embedding ❌

bank → [same vector]

Contextual Embedding ✅

bank + financial context → vector A

bank + river context → vector B

□ Easy Memory Trick

Static = Same
Contextual = Changes with Context

⚠ Interview Follow-up

Interviewer: “How does BERT generate contextual embeddings?”

Good answer:

“BERT uses the Transformer architecture and self-attention to look at the surrounding words. Because each word can attend to other words in the sentence, BERT can create a representation that depends on the complete context.”

Attention Mechanism

“Attention is a mechanism that allows a model to focus on the most important words in a sentence when understanding a particular word.

The main problem with traditional sequence models like RNNs is that they may struggle to remember important information when the sentence is long. Attention solves this by allowing the model to look at other words in the sequence and decide how much importance each word should receive.

For example, consider the sentence ‘The animal didn't cross the road because it was tired.’ To understand what ‘it’ refers to, the model needs to look at the surrounding words. Attention helps the model identify the relevant relationship between ‘it’ and ‘animal’.

In simple terms, attention answers the question: **‘Which words should I focus on to understand this word?’**”

□ Key Points to Memorize

Attention = Focus on important words

Remember:

- 👀 Looks at surrounding words
- 🎯 Assigns different importance to words
- □ Helps understand relationships
- 📜 Handles long-range dependencies better

- 🚀 Core idea behind **Transformers**

🔷 Small Example

Sentence:

“The dog chased the cat because it was angry.”

To understand **“it”**, the model needs to look at other words.

Attention helps determine which words are relevant to understanding **“it.”**

Think of attention like:

“When I am understanding one word, which other words should I pay attention to?”

□ Easy Memory Trick

Imagine you're reading a sentence with a **highlighter** 🖊.

You don't give equal importance to every word.

You highlight the words that help you understand the current word.

Attention = Smart highlighting of important words.

⚠ Very Important Interview Question

Interviewer: “Why do we need attention if we already have RNN/LSTM?”

Answer:

“RNNs process information sequentially, which can make it difficult to capture relationships between distant words. Attention allows the model to directly focus on relevant words, even when they are far apart, making it easier to capture long-range dependencies.”

Query, Key, and Value (Q, K, V)

“In the attention mechanism, every token is represented using three vectors: Query, Key, and Value.

Query represents what information the current word is looking for. Key represents what information each word contains or what it can be matched against. Value contains the actual information that will be used to create the final output.

The model compares the Query with the Keys using a similarity calculation, usually a dot product. This produces attention scores. After applying softmax, these scores become weights. The model then uses these weights to combine the Values.

In simple terms, Query asks ‘What am I looking for?’, Key says ‘What information do I have?’, and Value provides ‘Here is the actual information.’”

□ Key Points to Memorize

Q = What am I looking for? 🔍

K = What do I contain? 🏷️

V = What information should I provide? 📦

The basic flow:

Query + Keys → Attention Scores → Weights → Weighted Values → Output

🔷 Small Example

Sentence:

"The animal crossed the road because it was tired."

When the model processes **"it"**:

- **Query:** What should "it" refer to?
- **Keys:** Information from words like animal, road, crossed, etc.
- **Values:** The actual information associated with those words.

The model gives higher attention to the relevant word/context.

🧠 Easy Real-Life Example

Imagine you're in a library:

Query: 🔍

"I need a book about NLP."

Key: 🏷️

Book category/title information.

Value: 📄

The actual book content.

The Query finds which Keys are relevant and then retrieves information from their Values.

⚠️ Very Important Interview Point

Don't say:

❌ "Query is the question, Key is the answer, Value is the answer."

Better:

✅ **Query searches/matches, Key determines relevance, Value provides the information.**

💬 Likely Follow-up

Interviewer: “How are attention scores calculated?”

Answer:

“Usually, we calculate the dot product between the Query and Key vectors. Higher similarity produces a higher attention score. Then we apply softmax to convert the scores into attention weights and use those weights to combine the Value vectors.”

□ One-Line Memory

Q finds → K matches → V gives

Self-Attention

“Self-attention is a mechanism where each word in a sentence looks at other words in the same sentence to understand its meaning and relationship with them.

In self-attention, each token is converted into Query, Key, and Value vectors. The Query of one token is compared with the Keys of all the tokens to calculate attention scores. These scores are converted into weights using softmax, and then the model takes a weighted combination of the Value vectors.

This allows the model to decide which words are more important for understanding each token.

For example, in the sentence ‘The animal crossed the road because it was tired’, when processing the word ‘it’, self-attention can give more importance to relevant words such as ‘animal’ based on their relationship.

The major advantage is that self-attention can capture relationships between words even when they are far apart in a sentence.”

□ Key Points to Memorize

Remember this **5-step flow**:

1□Token → Q, K, V

2□Q × K → Attention Score

3□Softmax → Attention Weights

4□Weights × V → Weighted Values

5□Combine → Output

◈ Small Example

“The dog chased the cat because it was fast.”

When understanding **“it”**, the model checks other words:

it → dog?

it → cat?

it → fast?

Self-attention calculates how strongly **“it”** should pay attention to each word.

□ Easy Memory Trick

Imagine every word has a **spotlight** 🔦.

Each word asks:

“Which other words should I look at to understand myself?”

That's **Self-Attention**.

⚠ Self-Attention vs Attention

A common interview confusion:

Attention: Can involve information from different sequences.

Self-Attention:

Q, K, and V come from the same sequence.

For example, in:

“I love NLP.”

The tokens in this same sentence attend to one another.

💬 Very Likely Follow-up

Interviewer: “Can you explain the mathematical formula for self-attention?”

The expected answer is:

“Self-attention is calculated using Query, Key, and Value. We take Q multiplied by the transpose of K, scale it, apply softmax, and multiply the result with V.”

In short:

QK^T → Scale → Softmax → × V

Scaled Dot-Product Attention

“Scaled Dot-Product Attention is the mechanism used to calculate attention weights between Query, Key, and Value vectors in a Transformer.

First, we calculate the dot product between the Query and the Key vectors. This tells us how relevant each key is to the query. Then we divide the score by the square root of the Key vector dimension, which is called scaling.

After scaling, we apply Softmax to convert the scores into attention weights. Finally, we multiply these weights with the Value vectors to get the final attention output.

The scaling step is important because when the vector dimension is large, the dot-product values can become very large. This can make Softmax produce extremely small gradients and make training difficult. Dividing by the square root of the dimension helps keep the values stable."

□ Key Points to Memorize

Remember this flow:

$Q \times K^T \rightarrow$ Divide by $\sqrt{d_k} \rightarrow$ Softmax $\rightarrow \times V$

That's the **most important interview point**. 🔥

🔷 Small Example

Suppose Query compares with three Keys:

Scores = [2, 1, 0]

Softmax converts them approximately into:

[0.67, 0.24, 0.09]

These are the **attention weights**.

So the model gives:

- Key 1 → **67% importance**
- Key 2 → **24% importance**
- Key 3 → **9% importance**

Then these weights are used to combine the corresponding **Value vectors**.

□ Easy Memory Trick

Think:

Score → Stabilize → Focus → Collect

- **Score:** $Q \times K^T$
- **Stabilize:** divide by $\sqrt{d_k}$
- **Focus:** Softmax
- **Collect:** weighted Values

⚠ ☆ Why divide by $\sqrt{d_k}$?

This is a **favorite interviewer question**.

Answer:

"We divide by $\sqrt{d_k}$ to prevent the dot-product scores from becoming too large when the vector dimension increases. Large scores can make Softmax very sharp and lead to very small gradients. Scaling keeps training more stable."

Multi-Head Attention

"Multi-Head Attention is an extension of self-attention where the model uses multiple attention heads instead of a single attention operation.

Each attention head learns to focus on different relationships or patterns in the input. For example, one head may focus on grammatical relationships, another may focus on relationships between subjects and objects, and another may focus on nearby words.

Each head independently calculates Query, Key, and Value and produces an attention output. Then all the outputs are concatenated and passed through a final linear layer.

The main advantage is that the model can learn different types of relationships at the same time, which helps it understand language more effectively."

□ Key Points to Memorize

Multi-Head Attention = Multiple attention mechanisms working in parallel.

Remember the flow:

Input → Multiple Heads → Attention → Concatenate → Linear Layer → Output

🔷 Small Example

Sentence:

"The boy played cricket because he was excited."

Different attention heads might learn different relationships:

Head 1:
he ↔ boy → identifies who **he** refers to.

Head 2:
played ↔ cricket → identifies the action and object.

Head 3:
excited ↔ because → captures another relationship.

👉 Multiple heads allow the model to look at the sentence from **different perspectives**.

□ Easy Memory Trick

Imagine **one person looking at a sentence** 🧍 versus **8 people looking at it from different angles** 👥.

One head = One perspective
Multi-head = Multiple perspectives

⚠ **Very Important Interview Question**

Interviewer: "Why not just use one large attention layer?"

Answer:

"Multiple heads allow the model to learn different relationships in different representation spaces simultaneously. This gives the model richer information than relying on a single attention pattern."

💬 **Another Common Question**

Interviewer: "What happens after all attention heads produce their outputs?"

Answer:

"The outputs of all heads are concatenated and passed through a linear projection layer to produce the final Multi-Head Attention output."

□ One-Line Memory

Multi-Head Attention = Different heads learn different relationships → Combine their information.

Positional Encoding

"Positional Encoding is a technique used in Transformers to give the model information about the position or order of tokens in a sentence.

Self-attention can look at all tokens at the same time, but by itself it doesn't know whether a word comes first, second, or third. Positional encoding solves this problem by adding position information to the token embeddings.

For example, in the sentences 'Dog bites man' and 'Man bites dog', the same words are present but their order changes the meaning. Positional information helps the Transformer distinguish these different arrangements.

In the original Transformer, positional encoding was created using sine and cosine functions. Modern Transformer models can also use other approaches such as learned positional embeddings or rotary positional embeddings."

□ Key Points to Memorize

Positional Encoding = Gives word-order information 🔑

Remember:

- Transformer processes tokens **in parallel**
- Self-attention alone doesn't inherently know **position**
- Positional information is added to token representations
- Original Transformer → **Sine & Cosine**

- Modern models may use other methods

🔷 Small Example

Compare:

“Dog bites man.”

vs.

“Man bites dog.”

Same words ❗

Different order → **Different meaning**.

Positional information tells the model:

Dog = Position 1
bites = Position 2
man = Position 3

So the model can understand the order.

□ Easy Memory Trick

Think of a **queue** □□□:

Without position:

A, B, C

The model knows the people but not necessarily their order.

With position:

A → 1
B → 2
C → 3

Now it knows **who comes where**.

⚠ ☆ Common Interview Question

Interviewer: “Why doesn't self-attention automatically understand word order?”

Answer:

“Because self-attention processes all tokens together and calculates relationships between them, but it doesn't inherently encode the sequence order. Positional information is therefore added to tell the model where each token occurs.”

□ One-Line Memory

Attention tells the model WHAT is related; positional encoding tells it WHERE the token is.

Transformer Architecture

“Transformer is a neural network architecture introduced in the paper ‘Attention Is All You Need’. It is mainly based on the attention mechanism and does not rely on RNNs or CNNs to process sequences.

The original Transformer has two main parts: an **Encoder** and a **Decoder**. The Encoder processes the input and creates contextual representations, while the Decoder uses those representations to generate the output sequence.

Each Encoder and Decoder layer contains important components such as **Multi-Head Attention and a Feed-Forward Neural Network**, along with **residual connections and layer normalization**.

The Transformer also uses positional information because attention itself does not inherently understand the order of tokens.

Transformers became very important in NLP because they can process tokens in parallel and capture relationships between words even when they are far apart. Modern models such as BERT and GPT are based on Transformer architecture.”

□ Key Points to Memorize

Original Transformer:

Encoder + Decoder

Encoder:

Self-Attention → Add & Norm → Feed Forward → Add & Norm

Decoder:

Masked Self-Attention → Add & Norm → Cross-Attention → Add & Norm → Feed Forward → Add & Norm

Don't worry about memorizing every layer immediately. First remember:

Attention + Feed Forward + Residual + LayerNorm

◈ Small Example

Suppose we want to translate:

English: “I love India”

to

French: “J'aime l'Inde”

Encoder:

Reads and understands:

I → love → India

Decoder:

Uses that information to generate:

J'aime → l'Inde

□ Easy Memory Trick

Think of a Transformer like a **translator team**:

Encoder = Understand the input □

Decoder = Generate the output ✍

⚠ ☆ Very Important Interview Question

Interviewer: "Why are Transformers better than RNNs for many NLP tasks?"

Good answer:

"Transformers can process tokens in parallel instead of processing them strictly one by one like RNNs. They also use self-attention to directly capture relationships between distant tokens. This makes them easier to parallelize during training and effective at learning long-range dependencies."

🔥 One-Line Memory

Transformer = Attention-based architecture that understands relationships between tokens and processes them efficiently in parallel.

Encoder vs Decoder

"In the original Transformer architecture, the Encoder and Decoder have different responsibilities.

The **Encoder** takes the input sequence and converts it into contextual representations. It uses self-attention, so each token can understand its relationship with other input tokens.

The **Decoder** takes the encoder's representations and generates the output sequence step by step. It uses masked self-attention so that while generating a token, it cannot look at future tokens. It can also use cross-attention to focus on the encoder's output.

So, in simple terms, the Encoder is mainly responsible for **understanding the input**, while the Decoder is responsible for **generating the output**."

□ Key Points to Memorize

Encoder → Understand 🕮

Decoder → Generate ✍

Encoder	Decoder
Understands input	Generates output
Self-attention	Masked self-attention
Creates contextual representations	Generates tokens
Cannot see future output — not applicable	Cannot see future tokens
Example: BERT	Example: GPT

🔷 Small Example

For translation:

Input: "I love India"

Encoder:

Understands the meaning and relationships between I, love, and India.

Decoder:

Generates the translated sentence one token at a time.

J'aime → l'Inde

□ Easy Memory Trick

Think about a translator:

Encoder = Reads & Understands □

Decoder = Writes the Translation ✍

⚠ ☆ Important Interview Question

Interviewer: "Does GPT use both Encoder and Decoder?"

Answer:

"No. GPT is a decoder-only Transformer model. It mainly uses masked self-attention to generate text from previous tokens. BERT, on the other hand, is an encoder-only Transformer model."

🔥 BERT vs GPT — Remember This

BERT → Encoder-only → Understanding

GPT → Decoder-only → Generation

Original Transformer → Encoder + Decoder → Sequence-to-sequence tasks

□ One-Line Memory

Encoder understands; Decoder generates.

BERT

BERT = Bidirectional Encoder Representations from Transformers

“BERT stands for Bidirectional Encoder Representations from Transformers. It is a Transformer-based language model developed by Google, and it uses the Encoder part of the Transformer.

The important feature of BERT is that it understands a word by looking at the context on both sides of the word. This is called **bidirectional context**.

BERT is pre-trained on a large amount of text and then can be fine-tuned for specific NLP tasks such as sentiment analysis, text classification, named entity recognition, and question answering.

One of BERT's important pre-training techniques is **Masked Language Modeling**, where some words are hidden and the model learns to predict the missing words. BERT was also originally trained with **Next Sentence Prediction**.

In simple terms, BERT is mainly designed for **understanding language and context**, rather than generating text like GPT.”

□ Key Points to Memorize

Remember these **5 points**:

1. **BERT = Encoder-only**
2. **Bidirectional context**
3. **Pre-trained on large text**
4. **Masked Language Modeling**
5. **Used for understanding tasks**

◈ Small Example

Sentence:

“The bank was crowded because people wanted to deposit money.”

BERT looks at the words **before and after “bank”**.

So it understands:

bank = financial institution 🏦

rather than river bank.

□ Easy Memory Trick

BERT = Read the sentence from BOTH directions ↔

It uses context from:

Left → Right

and

Right → Left

to build a contextual representation.

⚠ ☆ Very Important: BERT vs GPT

BERT	GPT
Encoder-only	Decoder-only
Mainly language understanding	Mainly text generation
Bidirectional context	Causal/left-to-right context
Masked Language Modeling	Next-token prediction
Classification, NER, QA	Chat, completion, generation

💬 Common Interview Question

Interviewer: "Why is BERT called bidirectional?"

Answer:

"Because during its pre-training, BERT uses information from both the left and right context of a token to build its representation. This allows it to understand a word based on the complete surrounding context."

🔥 One-Line Memory

BERT = Encoder-only Transformer + Bidirectional Context + Masked Language Modeling.

Masked Language Modeling (MLM)

"Masked Language Modeling is one of the main pre-training techniques used by BERT. In MLM, some tokens in a sentence are hidden or masked, and the model is trained to predict those missing tokens using the surrounding context.

For example, if the sentence is 'I love machine learning', we can mask the word 'machine', giving the model 'I love [MASK] learning'. BERT uses the context on both sides to predict the missing word.

This is different from traditional next-word prediction because BERT can use both left and right context. By doing this on a large amount of text, BERT learns useful representations of language and word relationships.

After pre-training, BERT can be fine-tuned for tasks such as sentiment analysis, classification, named entity recognition, and question answering.”

□ Key Points to Memorize

MLM = Hide → Understand Context → Predict

Remember:

- 🔒 Some tokens are **masked**
- □ Model uses surrounding context
- 🎯 Predicts the missing token
- ↔ Uses **both left and right context**
- 🤖 Major pre-training objective of **BERT**

🔷 Small Example

Original:

“I love machine learning.”

Masked:

“I love [MASK] learning.”

BERT predicts:

machine ✅

Another example:

“The cat is sitting on the [MASK].”

Possible prediction:

mat 🐈

□ Easy Memory Trick

Think of a **fill-in-the-blank exam** 📝:

“I am learning ___.”

The surrounding words help you guess the missing word.

That's essentially the idea behind **Masked Language Modeling**.

⚠ ☆ Important Interview Question

Interviewer: “Why does BERT use masking instead of simply predicting the next word?”

Answer:

“Because BERT is designed to learn bidirectional context. Masking allows the model to use information from both the left and right sides of the missing token, whereas next-token prediction mainly uses previous tokens.”

💬 **One More Important Point**

Don't confuse:

MLM:

Predict a **masked/missing token**.

Next-token prediction:

Predict the **next token** based on previous tokens.

👉 **BERT → MLM**

👉 **GPT → Next-token prediction**

□ One-Line Memory

“BERT learns by hiding words and predicting them from both sides of the context.”

BERT Fine-Tuning

“Fine-tuning means taking a pre-trained BERT model and training it further on a smaller, task-specific dataset.

BERT is first pre-trained on a large amount of text using objectives such as Masked Language Modeling. During fine-tuning, we take this pre-trained model and adapt it to a specific task, such as sentiment analysis, text classification, named entity recognition, or question answering.

Usually, we add a task-specific output layer on top of BERT and train the model using our labeled dataset. We update BERT's parameters along with the new layer, although in some cases we may freeze some layers.

For example, if we want to build a movie-review sentiment classifier, we can take pre-trained BERT, add a classification layer, and fine-tune it using reviews labeled as positive or negative.

The advantage is that we don't have to train a language model from scratch. We can reuse the knowledge BERT learned during pre-training.”

□ Key Points to Memorize

Pre-trained BERT → Add task layer → Train on task data → Fine-tuned model

Remember:

- 🗂️ **Pre-training** → General language knowledge
- 🎯 **Fine-tuning** → Specific task

- 🏷️ Usually uses **labeled data**
- ⚡ Much easier than training from scratch

🔷 Small Example

Goal:

Sentiment Analysis

Dataset:

Review	Label
“Amazing movie!”	Positive
“Very boring.”	Negative

Process:

Pre-trained BERT

⬇️

Add classification layer

⬇️

Fine-tune with movie reviews

⬇️

Sentiment classifier ✅

🧠 Easy Memory Trick

Pre-training = Learn generally 📚
Fine-tuning = Specialize 🎯

Think:

BERT already knows language → We teach it our specific job.

⚠️ ☆ Common Interview Question

Interviewer: “Why don't we train BERT from scratch?”

Good answer:

“Training BERT from scratch requires a huge dataset, significant computational resources, and a lot of time. Fine-tuning a pre-trained BERT model is much more efficient because the model already has general language knowledge.”

💬 Important Distinction

Pre-training:

Large data + general language learning

Fine-tuning:

Smaller task-specific data + specific objective

□ One-Line Memory

Fine-tuning = Taking a pre-trained model and adapting it to a specific task.

GPT

GPT = Generative Pre-trained Transformer

"GPT stands for Generative Pre-trained Transformer. It is a Transformer-based language model designed mainly for generating text. GPT uses the decoder part of the Transformer architecture and uses causal or masked self-attention, which means the model can only look at the previous tokens when predicting the next token.

GPT is first pre-trained on a large amount of text using a next-token prediction objective. For example, if the input is 'I love machine', the model learns to predict the next token, such as 'learning'.

Because GPT learns language patterns from huge amounts of text, it can perform tasks such as text generation, summarization, question answering, translation, coding, and conversational AI.

In simple terms, GPT reads the previous context and predicts what should come next, one token at a time."

□ Key Points to Memorize

Remember **5 things**:

1. **GPT = Generative Pre-trained Transformer**
2. **Decoder-only Transformer**
3. **Causal/Masked Self-Attention**
4. **Next-token prediction**
5. Mainly used for **text generation**

◈ Small Example

Input:

"The capital of India is"

GPT predicts:

"New"

Then:

“Delhi”

So generation happens approximately:

Input → Predict next token → Add it → Predict next token → Repeat

□ Easy Memory Trick

GPT = Predict the Next Token → Generate Text

Think of it like autocomplete on steroids:

“I am going to…”

Model predicts:

“the…”

Then:

“market…”

Then:

“today…”

BERT vs GPT

BERT	GPT
Encoder-only	Decoder-only
Mainly understanding	Mainly generation
Masked Language Modeling	Next-token prediction
Bidirectional context during pretraining	Previous-token context
Classification, NER, QA	Chat, writing, generation

💬 **Very Important Interview Question**

Interviewer: “Why is GPT called autoregressive?”

Answer:

“Because GPT generates text one token at a time, and each newly generated token becomes part of the context used to predict the next token.”

Example:

I → I love → I love NLP → I love NLP because...

Each prediction depends on the previous tokens.

□ One-Line Memory

GPT = Decoder-only Transformer that learns to predict the next token and generate text.

BERT vs GPT

"BERT and GPT are both based on the Transformer architecture, but they are designed for different purposes.

BERT uses the **Encoder** part of the Transformer and is mainly designed for understanding language. During pre-training, BERT uses Masked Language Modeling, where it predicts missing tokens using context from both sides.

GPT uses the **Decoder** part of the Transformer and is mainly designed for generating text. It is trained using next-token prediction, where it predicts the next token based on the previous tokens.

So, BERT is generally better suited for understanding and representation tasks such as classification and named entity recognition, while GPT is designed for text generation and conversational applications."

□ Key Points to Memorize

Feature	**BERT**	**GPT**
Architecture	Encoder-only	Decoder-only
Main purpose	Understanding	Generation
Training objective	Masked Language Modeling	Next-token prediction
Context	Bidirectional during pretraining	Previous-token/causal
Generation	Not its primary purpose	Yes
Examples	Classification, NER	Chat, writing, coding

◈ Small Example

Sentence:

"I went to the bank to deposit money."

BERT:

Understands what **bank** means from the surrounding context.

☞ Understanding

GPT:

Given:

“I went to the bank to…”

It predicts likely next tokens such as:

“deposit money.”

👉 **Generation**

□ Easy Memory Trick

BERT = Understand □
GPT = Generate ✍

And:

BERT → Encoder
GPT → Decoder

⚠ ☆ **Common Interview Trap**

Interviewer: “Is BERT always better than GPT for NLP?”

Don't say **yes**.

Say:

“It depends on the task. BERT is well suited for many language understanding tasks, while GPT-style decoder models are well suited for generation. The right architecture depends on the application.”

💬 **One-Line Memory**

“BERT understands context; GPT uses context to generate the next token.”

RNN — Recurrent Neural Network

“RNN stands for Recurrent Neural Network. It is a type of neural network designed to process sequential data, where the order of information is important.

In NLP, RNNs can process a sentence one word or token at a time. At each step, the RNN takes the current input and the hidden state from the previous step. The hidden state acts like a memory and carries information from previous words.

For example, while processing the sentence ‘I love machine learning’, the RNN processes ‘I’, then ‘love’, then ‘machine’, and finally ‘learning’, carrying information from previous words at each step.

However, standard RNNs have difficulty remembering information over long sequences because of problems such as the **vanishing gradient problem**. This led to improved architectures like LSTM and GRU, and eventually attention-based Transformers.”

□ Key Points to Memorize

RNN = Sequence + Memory + Previous Information

Remember:

- 🔁 Processes sequence **step by step**
- 🧠 Uses a **hidden state**
- 📚 Previous information influences current output
- ❌ Difficult with **long-term dependencies**
- ❌ Can suffer from **vanishing gradients**
- ➡️ Led to **LSTM/GRU → Attention → Transformers**

🔷 Small Example

Sentence:

"I love NLP."

RNN processes:

I → love → NLP

At each step:

Current word + Previous hidden state → New hidden state

So when processing NLP, the model has information carried from I and love.

🧠 Easy Memory Trick

Think of a person reading a sentence **one word at a time** and remembering what they read:

Read → Remember → Read → Remember → Read

That's the basic idea of an RNN.

⚠️ ☆ Common Interview Question

Interviewer: "What is the main limitation of RNN?"

Answer:

"The main limitation is difficulty in learning long-term dependencies. During backpropagation through time, gradients can become extremely small, causing the vanishing gradient problem. As a result, the model may forget information from earlier parts of a long sequence."

🔥 RNN → LSTM → Transformer

Remember this evolution:

RNN
⬇️ Problem: long-term memory

LSTM / GRU
↓ Better memory

Attention
↓ Focus on relevant words

Transformer
↓ Parallel processing + self-attention

□ One-Line Memory

RNN = Processes sequences step-by-step while carrying information through a hidden state.

LSTM — Long Short-Term Memory

"LSTM stands for Long Short-Term Memory. It is an improved type of RNN designed to handle long-term dependencies better than a standard RNN.

The main problem with RNNs is that they can forget information from earlier parts of a long sequence because of the vanishing gradient problem. LSTM addresses this using a special memory mechanism called the **cell state** and three gates: **Forget Gate, Input Gate, and Output Gate**.

The Forget Gate decides what information should be removed, the Input Gate decides what new information should be stored, and the Output Gate decides what information should be passed to the next step.

For example, in a long sentence, LSTM can preserve an important subject from an earlier part of the sentence and use it later when needed.

So, the main advantage of LSTM over a basic RNN is its ability to remember important information for a longer period."

□ Key Points to Memorize

LSTM = RNN + Better Memory □

Remember the **3 Gates**:

1. **Forget Gate** → What should I forget?
2. **Input Gate** → What new information should I store?
3. **Output Gate** → What should I output?

And:

Cell State = Long-term memory

◈ Small Example

Consider:

“I grew up in India... many words later... I speak Telugu.”

An RNN may struggle to retain the earlier information.

LSTM can use its memory mechanism to retain important information and use it later.

□ Easy Memory Trick

Think of LSTM as a **smart notebook** 📓:

- **Forget Gate** → Erase unnecessary information 🗑️
- **Input Gate** → Write important new information ✍️
- **Output Gate** → Read out what is needed 👀

⚠ ☆ Very Important Interview Question

Interviewer: “How does LSTM solve the vanishing gradient problem?”

Good answer:

“LSTM uses a cell state and gating mechanism that provides a better path for information and gradients to flow across many time steps. This helps preserve important information and reduces the vanishing-gradient problem compared with a standard RNN.”

Don't say:

✕ “LSTM completely eliminates vanishing gradients.”

Better:

✅ **“LSTM significantly reduces the problem.”**

RNN vs LSTM

RNN	LSTM
Simple memory	Better long-term memory
One hidden state	Hidden state + cell state
Can forget older information	Better at retaining important information
More affected by vanishing gradients	Reduces the problem
Simpler	More complex

🔥 **One-Line Memory**

RNN has memory; LSTM has controlled memory using gates.

LSTM Gates

“LSTM uses three gates to control the flow of information through the network: the Forget Gate, Input Gate, and Output Gate.

The **Forget Gate** decides which information from the previous cell state should be removed. For example, if old information is no longer useful, the model can forget it.

The **Input Gate** decides what new information should be stored in the cell state. It identifies which parts of the current input are important.

The **Output Gate** decides what information from the cell state should be used as the output or hidden state for the current time step.

Together, these gates allow LSTM to selectively forget, remember, and output information, which helps it handle long-term dependencies better than a standard RNN.”

□ Key Points to Memorize

Just remember:

1. Forget Gate

“What should I forget?”

↓

2. Input Gate

“What should I remember?”

↓

3. Output Gate

“What should I show?”

That's enough for a first-round interview.

◈ Small Example

Suppose the model reads:

“I live in Hyderabad. I moved to Bangalore last year.”

When processing the second sentence:

Forget Gate:
May reduce the importance of old location information.

Input Gate:
Stores the new important information:

Bangalore

Output Gate:
Decides how much of this information should be passed forward.

□ Easy Memory Trick

Imagine your **phone's Notes app** 📱:

- 🗑️ **Forget Gate** → Delete old/unnecessary notes
- ✍️ **Input Gate** → Save new important notes
- 📤 **Output Gate** → Share the required note

⚠️ ☆ Common Interview Confusion

Question: "Does the Forget Gate delete the entire memory?"

❌ No.

It controls **how much of each piece of information should be forgotten**.

For example:

0 → forget most/all of that information
1 → keep most/all of that information

The gate values are generally between **0 and 1** because sigmoid activation is used.

💬 Likely Follow-up

Interviewer: "What is the difference between hidden state and cell state?"

Remember:

Cell State → Long-term information □

Hidden State → Current/output information 📤

LSTM vs GRU

"LSTM and GRU are both improved versions of RNNs designed to handle long-term dependencies and reduce the vanishing gradient problem.

The main difference is their architecture. **LSTM has three gates — Forget, Input, and Output — along with a separate cell state. GRU is simpler and combines some of these mechanisms into two gates: Update and Reset. GRU does not have a separate cell state; it uses the hidden state to store information.**

Because GRU has fewer gates and fewer parameters, it is generally simpler and faster to train than LSTM. However, LSTM provides more control over memory, which can be useful for some complex sequence tasks.

In practice, the choice depends on the dataset, task, and computational requirements."

□ Key Points to Memorize

LSTM

3 Gates + Cell State

- Forget Gate
- Input Gate
- Output Gate
- Separate Cell State

GRU

2 Gates + Hidden State

- Update Gate
- Reset Gate
- No separate Cell State

🔷 Small Example

Suppose we're processing:

"I studied Python for two years, and now I work as a..."

Both LSTM and GRU can remember useful information from earlier words to help predict what comes next.

But:

LSTM → More detailed memory control

GRU → Simpler and faster

📌 Quick Comparison

Feature	LSTM	GRU
Gates	3	2
Cell state	✅ Separate	❌ No
Hidden state	✅	✅
Parameters	More	Fewer
Training	Usually slower	Usually faster
Architecture	More complex	Simpler

□ Easy Memory Trick

LSTM = More control
GRU = Less complexity ⚡

Or:

LSTM → 3 Gates + Cell State
GRU → 2 Gates + Hidden State

⚠ ☆ Interview Question

Interviewer: "Which is better, LSTM or GRU?"

Don't say one is always better.

Say:

"Neither is universally better. GRU is simpler and computationally cheaper, while LSTM provides more explicit control over memory. I would choose based on the task, dataset, and performance requirements, and ideally compare them experimentally."

🔥 One-Line Memory

"LSTM is more complex with 3 gates and a cell state; GRU is simpler with 2 gates and no separate cell state."

Vanishing Gradient & Exploding Gradient

"Vanishing and exploding gradients are problems that can occur during backpropagation, especially in deep networks and recurrent neural networks.

In the **vanishing gradient problem**, gradients become extremely small as they are propagated backward through many layers or time steps. Because of this, earlier parts of the network learn very slowly or may stop learning. In RNNs, this can cause difficulty in remembering long-term dependencies.

In the **exploding gradient problem**, gradients become extremely large during backpropagation. This can cause unstable training and very large updates to the model parameters.

Techniques such as LSTM and GRU help reduce the vanishing-gradient problem, while **gradient clipping** is commonly used to control exploding gradients."

Key Points to Memorize

Vanishing Gradient 📉

Gradient → Very Small → Earlier layers learn slowly

Result:

❌ Difficult to learn long-term dependencies.

Exploding Gradient 📈

Gradient → Very Large → Huge parameter updates

Result:

❌ Unstable training.

🔷 **Small Example**

Imagine passing a signal through many steps.

Vanishing:

1 → 0.5 → 0.1 → 0.01 → 0.001

The signal becomes almost **zero**.

Exploding:

1 → 2 → 10 → 100 → 1000

The signal becomes extremely **large**.

🧠 **Easy Memory Trick**

Vanishing = Gradient disappears 👻

Exploding = Gradient becomes huge 💥

⚠️ ⭐ **Interview Question**

Interviewer: "How do you handle exploding gradients?"

Answer:

"A common solution is gradient clipping. We set a maximum allowed gradient norm or value and clip gradients when they become too large. This prevents extremely large parameter updates and makes training more stable."

💬 **"How do you handle vanishing gradients?"**

Good answer:

"We can use architectures such as LSTM or GRU, which provide better paths for information and gradients to flow. Appropriate activation functions and initialization methods can also help."

🧠 **One-Line Memory**

Vanishing → Too small → Can't learn
Exploding → Too large → Unstable training

Sequence-to-Sequence (Seq2Seq)

"Sequence-to-Sequence, or Seq2Seq, is a neural network architecture used when both the input and output are sequences.

A typical Seq2Seq model has two main parts: an **Encoder** and a **Decoder**. The Encoder reads the input sequence and converts it into a representation. The Decoder then uses that representation to generate the output sequence, usually one token at a time.

Seq2Seq models were commonly used for tasks such as machine translation, text summarization, and conversational systems.

For example, in English-to-French translation, the Encoder reads ‘I love you’ and creates a representation of its meaning. The Decoder then generates the French translation ‘Je t'aime’.

A limitation of basic Seq2Seq models is that compressing a long input sequence into a single fixed representation can lose information. This problem motivated the use of **Attention**, which allows the Decoder to focus on relevant parts of the input.”

□ Key Points to Memorize

Seq2Seq = Sequence → Sequence

Main components:

Encoder → Understand input

Decoder → Generate output

Common applications:

- 🌍 Machine Translation
- 📝 Text Summarization
- 💬 Chatbots
- ❓ Question Answering

🔷 Small Example

Input:

I love NLP

Encoder:

Understands the input.



Decoder:

Generates another sequence.

Output:

J'aime le NLP

□ Easy Memory Trick

Think of a **translator**:

📖 **Encoder = Reads**
✍️ **Decoder = Writes**

⚠️ ☆ Important Interview Question

Interviewer: “What is the problem with the basic Seq2Seq model?”

Answer:

“The basic encoder-decoder model may compress the entire input sequence into a fixed-size representation. For long sequences, this can cause information loss. Attention solves this by allowing the decoder to focus on different relevant parts of the input.”

🔥 Evolution to Remember

This is **very useful for interviews**:

RNN
↓
LSTM/GRU
↓
Seq2Seq Encoder-Decoder
↓
Attention
↓
Transformer
↓
BERT / GPT / Modern LLMs

□ One-Line Memory

Seq2Seq = Encoder converts input sequence into useful representation → Decoder generates output sequence.

Attention in Seq2Seq

“Attention was introduced in Seq2Seq models to solve the information bottleneck problem of the basic encoder-decoder architecture.

In a basic Seq2Seq model, the Encoder tries to compress the entire input sequence into a single fixed-length representation. For long sentences, some important information can be lost.

Attention solves this by allowing the Decoder to look at all the Encoder's hidden states and assign different importance to them while generating each output token.

For example, during translation, when the Decoder is generating a particular French word, it can focus more on the relevant English words instead of relying only on one fixed representation.

So, attention allows the model to dynamically focus on different parts of the input depending on what it is currently generating."

□ Key Points to Memorize

Basic Seq2Seq:

Entire input → **One fixed representation** ✕

Seq2Seq + Attention:

Decoder → **Looks at all relevant encoder states** ☑

Remember:

Attention = Dynamic Focus 🎯

◈ Small Example

English:

"I love eating pizza."

While generating the translation:

Decoder generating "pizza"

It can give high attention to:

pizza 🍕

rather than equally focusing on:

I, love, eating

For another output word, the attention can shift to different input words.

□ Easy Memory Trick

Imagine translating a sentence using a **highlighting pen** 🖊.

Instead of remembering the whole sentence as one thing, you highlight the words you need **for each output word**.

Different output → Different focus

⚠ ☆ Common Interview Question

Interviewer: "What problem does Attention solve?"

Perfect answer:

“Attention reduces the fixed-length information bottleneck in basic Seq2Seq models. Instead of forcing the decoder to depend on one fixed representation of the entire input, it can access and focus on relevant encoder hidden states at each decoding step.”

💬 **Important Connection**

Don't confuse **Seq2Seq Attention** with **Self-Attention**:

Seq2Seq Attention:

Decoder looks at **Encoder outputs**.

Self-Attention:

Tokens within the **same sequence** look at each other.

This distinction is frequently asked in interviews. 🔥

□ One-Line Memory

“Attention lets the decoder look at the right parts of the input instead of relying on one fixed representation.”

Cross-Attention

“Cross-attention is an attention mechanism where one sequence attends to information from another sequence. In a Transformer encoder-decoder model, the Decoder uses cross-attention to look at the Encoder's output while generating the target sequence.

In cross-attention, the **Query comes from the Decoder**, while the **Key and Value come from the Encoder output**. The Decoder uses this information to determine which parts of the input are most relevant for generating the next output token.

For example, during English-to-French translation, when the Decoder is generating a French word, it can use cross-attention to focus on the relevant English words produced by the Encoder.

So, cross-attention basically allows the Decoder to ask the Encoder: **‘Which information from the input is useful for generating my current output?’**”

□ Key Points to Memorize

The **most important point**:

Q → Decoder
K → Encoder
V → Encoder

🔷 **Small Example**

Translation:

English: “I love India.”

Encoder:

Processes the English sentence.

Decoder:

Generates the French translation.

While generating a particular output token:

Decoder Query 🔍
↓
Encoder Keys 🏷️
↓
Encoder Values 📦
↓
Relevant information → Decoder

□ Easy Memory Trick

Think:

Decoder asks → Encoder provides

Decoder:

"I need information about this part."

Encoder:

"Here is the relevant information."

⚠ ☆ Self-Attention vs Cross-Attention

Self-Attention	Cross-Attention
Q, K, V from the same sequence	Q from one sequence, K/V from another
Tokens attend to each other	One sequence attends to another
Used in Encoder	Used between Decoder and Encoder
Example: BERT	Example: Encoder-Decoder Transformer

💬 Very Likely Interview Question

Interviewer: "Does GPT use cross-attention?"

Answer:

"GPT is a decoder-only architecture, so standard GPT models do not have an encoder and therefore do not use encoder-decoder cross-attention. They mainly use masked self-attention."

□ One-Line Memory

Self-Attention = Same sequence talks to itself.
Cross-Attention = One sequence looks at another sequence.

Masked Self-Attention

"Masked self-attention is a type of self-attention used in decoder-based models like GPT to prevent the model from looking at future tokens while generating text.

In normal self-attention, a token can attend to other tokens in the sequence. But during text generation, if the model could see future tokens, it would already know the answer and training would not represent the real generation process.

Therefore, we use a causal mask, usually a triangular mask, that allows each token to attend only to itself and the tokens that came before it.

For example, in the sentence 'I love NLP', when predicting 'NLP', the model can look at 'I' and 'love', but it cannot look at any token that comes after 'NLP'.

This makes GPT autoregressive: it generates text one token at a time using only previous context."

□ Key Points to Memorize

Masked Self-Attention = No Looking Ahead 🚫👀

Remember:

- Used in **decoder-only models like GPT**
- Prevents access to **future tokens**
- Uses a **causal mask**
- Enables **autoregressive generation**
- Each token can attend to **itself + previous tokens**

🔷 Small Example

Input:

"I love NLP"

When predicting:

"love"

It can see:

I ✅
love ✅

But it cannot see:

NLP ❌

When predicting:

“NLP”

It can see:

I ✅
love ✅
NLP ✅

□ Easy Memory Trick

Imagine an exam where students can only see **previous answers**, not future answers. 📝

Previous tokens = Allowed ✅
Future tokens = Blocked ❌

⚠ ☆ Very Important Interview Question

Interviewer: “Why is masking required in GPT?”

Answer:

“GPT is trained to predict the next token. If it could see future tokens during training, there would be information leakage because the answer would already be available. The causal mask prevents this and ensures the model learns to predict using only previous tokens.”

🔥 BERT vs GPT Connection

BERT:

Masked Language Modeling → predicts hidden tokens using **both sides of context**

GPT:

Causal/next-token prediction → uses **only previous context**

Feed-Forward Neural Network (FFN) in Transformer

“What happens after Multi-Head Attention?”

“A Feed-Forward Neural Network, or FFN, is a fully connected neural network applied independently to each token after the attention layer.

The attention layer is mainly responsible for allowing tokens to exchange information and understand relationships with other tokens. The Feed-Forward Network then processes each token's representation and learns more complex patterns or transformations.

In the original Transformer, the FFN consists mainly of two linear layers with a non-linear activation function between them, such as ReLU.

Importantly, the same FFN is applied separately to every token. It does not directly mix information between different tokens; the attention layer handles that.

So, in simple terms, **attention helps tokens communicate with each other, while the FFN processes and transforms each token's information.**”

□ Key Points to Memorize

Remember:

Attention = Token communication 🗣️

FFN = Token processing □

Transformer block roughly:

Multi-Head Attention
↓
Add & LayerNorm
↓
FFN
↓
Add & LayerNorm

🔷 Small Example

Suppose we have:

I | love | NLP

Attention:

Each token can look at the other tokens and understand relationships.

Then **FFN:**

Processes each token's updated representation individually:

I → transformed I

love → transformed love

NLP → transformed NLP

□ Easy Memory Trick

Think of a classroom:

Attention = Students talk to each other 🗣️

FFN = Each student studies and processes what they learned 📚

⚠ ☆ Important Interview Question

Interviewer: “Does the FFN combine information between different tokens?”

Answer:

“No. The FFN is applied independently to each token. The attention layer is responsible for mixing information between tokens.”

💬 **Another Common Question**

Interviewer: “Why do we need FFN if we already have attention?”

Answer:

“Attention allows tokens to exchange contextual information, but the FFN provides additional non-linear transformation of each token's representation. Together, they allow the Transformer to learn richer patterns.”

□ One-Line Memory

“Attention mixes information across tokens; FFN transforms each token individually.”

Residual Connections & Layer Normalization

“Residual connections and Layer Normalization are used in Transformers mainly to make training deeper networks more stable and effective.

A **residual connection**, also called a skip connection, adds the original input of a layer to the output of that layer. Instead of learning only a completely new representation, the network learns an improvement or residual over the original input. This helps information and gradients flow through deeper networks.

Layer Normalization normalizes the activations within each token's representation. It helps keep the values in a more stable range, which makes training more stable and can improve convergence.

In a Transformer block, we commonly see an attention or FFN sublayer followed by a residual connection and normalization. In simple terms, residual connections help information flow, while LayerNorm helps stabilize the training.”

□ Key Points to Memorize

🩹 **Residual Connection**

Input + Layer Output → Final Output

👉 Helps **information and gradients flow**

📏 **Layer Normalization**

👉 Normalizes activations

👉 Helps **training stability**

🔹 **Small Example**

Suppose a layer receives:

Input = X

The layer produces:

Attention(X)

With residual connection:

X + Attention(X)

Then LayerNorm is applied.

So remember:

Original information + New information → Normalize

□ Easy Memory Trick

Think of a student learning something new:

Residual Connection:

"Don't forget what you already knew." □

LayerNorm:

"Keep your information balanced and stable." ⚖️

⚠ ☆ Interview Question

Interviewer: "Why do we use residual connections?"

Good answer:

"Residual connections provide a direct path for information and gradients through the network. This makes optimization easier and helps prevent degradation problems when the network becomes deeper."

💬 Another Common Question

Interviewer: "Why do we use Layer Normalization instead of Batch Normalization in Transformers?"

Good answer:

"Layer Normalization works independently for each sequence example and does not depend on batch statistics. This makes it more suitable for variable-length sequence models like Transformers."

□ One-Line Memory

Residual = Better information flow 📎
LayerNorm = Stable training 📏

Transformer Encoder Block

“A Transformer Encoder consists of multiple identical encoder layers stacked on top of each other. Each encoder layer mainly contains two sublayers: Multi-Head Self-Attention and a Feed-Forward Neural Network.

First, the input tokens are converted into embeddings and positional information is added. The embeddings then go through Multi-Head Self-Attention, where each token can attend to other tokens in the same sequence.

The attention output passes through a residual connection and Layer Normalization. Then it goes through the Feed-Forward Network, which independently transforms each token's representation. Another residual connection and Layer Normalization are applied.

Multiple encoder layers can be stacked, allowing the model to learn increasingly complex contextual representations.

BERT is an example of an encoder-only Transformer model.”

□ Key Points to Memorize

The basic Encoder flow:

Token Embedding + Position
↓
Multi-Head Self-Attention
↓
Add & LayerNorm
↓
Feed-Forward Network
↓
Add & LayerNorm
↓
Output

◈ Small Example

Input:

“The cat is sleeping.”

The encoder processes all tokens together.

Self-Attention:

cat ↔ sleeping

The ↔ cat

cat ↔ is

The model learns relationships between the words.

FFN:

Then each token's representation is further transformed.

After several encoder layers:

Each token has a **rich contextual representation**.

□ Easy Memory Trick

Remember:

Attention → Add & Norm → FFN → Add & Norm

That's the **core Transformer Encoder block**. 🔥

⚠ ☆ Interview Question

Interviewer: "What is the difference between a Transformer Encoder and Decoder?"

Answer:

"The Encoder uses self-attention to understand the input sequence. The Decoder uses masked self-attention for autoregressive generation and, in an encoder-decoder Transformer, also uses cross-attention to access the Encoder's output."

□ One-Line Memory

Transformer Encoder = Self-Attention + FFN + Residual Connections + LayerNorm

📌 Architecture Connection

You should now be able to connect:

Tokenization
→ Embeddings
→ Positional Information
→ Self-Attention
→ Multi-Head Attention
→ FFN
→ Residual + LayerNorm
→ Transformer Encoder
→ BERT

Transformer Decoder Block

"The Transformer Decoder is responsible for generating the output sequence. In the original encoder-decoder Transformer, each decoder layer has three main components: masked self-attention, cross-attention, and a feed-forward network.

First, **masked self-attention** allows each output token to attend only to previous tokens, so the model cannot see future tokens.

Second, **cross-attention** allows the decoder to look at the Encoder's output. Here, the Query comes from the Decoder, while the Key and Value come from the Encoder.

Finally, the output passes through a Feed-Forward Network. Residual connections and Layer Normalization are used around these sublayers to make training stable.

GPT is a decoder-only Transformer, so it uses the decoder-style masked self-attention but does not have the Encoder or encoder-decoder cross-attention found in the original Transformer."

□ Key Points to Memorize

Original Transformer Decoder:

1□Masked Self-Attention
→ Don't look at future tokens ⃠

2□Cross-Attention
→ Look at Encoder output 🔍

3□Feed-Forward Network
→ Process representations □

Plus:

Residual Connections + LayerNorm

🔷 Small Example

Translation:

Input: "I love India."

Encoder: Understands the English sentence.

Decoder:

First predicts:

J'aime

Then:

l'Inde

While generating each token:

Masked Self-Attention:
Looks at previously generated tokens.

Cross-Attention:
Looks at relevant information from the English input.

□ Easy Memory Trick

Think:

Masked Self-Attention = Look at my past 👈

Cross-Attention = Look at the Encoder 🔍

FFN = Process the information □

⚠ ☆ Very Important Interview Question

Interviewer: “Does GPT have cross-attention?”

Answer:

“No. GPT is decoder-only, so there is no separate Encoder output to attend to. GPT mainly uses masked self-attention and Feed-Forward Networks.”

📌 Encoder vs Decoder — Final Memory

Encoder:

Self-Attention → FFN

Original Transformer Decoder:

Masked Self-Attention → Cross-Attention → FFN

GPT:

Masked Self-Attention → FFN

BERT:

Self-Attention → FFN

□ One-Line Memory

Decoder = Masked Self-Attention + Cross-Attention + FFN

BERT vs GPT vs T5

“BERT, GPT, and T5 are all Transformer-based models, but they use different architectures and training objectives.

BERT is an encoder-only model mainly designed for understanding text. It uses bidirectional context and was originally pre-trained using Masked Language Modeling.

GPT is a decoder-only model mainly designed for generating text. It uses causal self-attention and learns by predicting the next token.

**T5, or Text-to-Text Transfer Transformer, uses both an Encoder and a Decoder. It converts different NLP tasks into a text-to-text format. For example, sentiment classification can be represented as input text ‘This movie is amazing’ and output text ‘positive’.

So, BERT is mainly for understanding, GPT is mainly for generation, and T5 can handle both understanding and generation using an encoder-decoder architecture.”

□ Key Points to Memorize

Model	Architecture	Main Idea
BERT	Encoder-only	Understanding
GPT	Decoder-only	Generation
T5	Encoder + Decoder	Text → Text

◈ Small Example

Task:

“The movie was excellent.”

BERT:

Could classify it:

Positive

GPT:

Could continue:

“The movie was excellent because…”

T5:

Can treat the task as:

Input: The movie was excellent. sentiment:
Output: positive

□ Easy Memory Trick

BERT = Read 🕮
GPT = Write ✍
T5 = Read + Write 🔁

Or:

BERT → Encoder
GPT → Decoder
T5 → Encoder + Decoder

⚠ ☆ Interview Question

Interviewer: “Why is T5 called Text-to-Text?”

Answer:

“Because T5 represents different NLP tasks in the same format: text as input and text as output. For example, translation, summarization, and classification can all be expressed as text-to-text problems.”

□ One-Line Memory

BERT understands, GPT generates, T5 converts one piece of text into another.

Token Embeddings vs Positional Embeddings

“A Transformer needs two types of information to understand an input sequence: what each token means and where each token is located. Token embeddings provide the meaning representation, while positional embeddings or positional encodings provide the position information.

A **token embedding** converts each token into a numerical vector that represents its learned features. For example, the tokens ‘I’, ‘love’, and ‘NLP’ each get their own vectors.

But self-attention doesn't inherently know the order of these tokens. So we add positional information to the token embeddings. This tells the model whether a token is in position 1, position 2, position 3, and so on.

The resulting representation is then given to the Transformer layers.”

□ Key Points to Memorize

Token Embedding = WHAT is the token?

Positional Embedding/Encoding = WHERE is the token?

Overall:

Transformer Input = Token Representation + Position Information

◈ Small Example

Sentence:

“I love NLP.”

Token embeddings:

I → Vector A

love → Vector B

NLP → Vector C

Position information:

I → Position 1

love → Position 2

NLP → Position 3

The Transformer receives a combination of both.

□ Easy Memory Trick

Imagine a classroom:

Token Embedding: 🧑‍🎓

Who is the student?

Position Information: 💺

Where is the student sitting?

The Transformer needs **both**.

⚠ ☆ Important Interview Question

Interviewer: “Why can't we use only token embeddings?”

Answer:

“Token embeddings tell the model what each token represents, but they don't tell the model the order of tokens. Since word order can change meaning, positional information is needed.”

Example:

“Dog bites man.”

vs.

“Man bites dog.”

Same words, different order → different meaning.

□ One-Line Memory

Token embedding tells WHAT; positional information tells WHERE.

📌 Our NLP Journey So Far

Raw Text
→ Tokenization
→ Token Embeddings
→ Positional Information
→ Self-Attention
→ Multi-Head Attention
→ FFN
→ Transformer
→ BERT / GPT

Fine-Tuning vs Prompting vs RAG

“Prompting, RAG, and fine-tuning are three different ways of adapting an LLM for a specific use case.

Prompting means giving the model clear instructions and context without changing the model's parameters. It is the simplest approach and is useful when the model already has the required knowledge.

RAG, or Retrieval-Augmented Generation, retrieves relevant information from an external knowledge source and provides it to the LLM before generating the answer. It is useful when we need answers based on private, domain-specific, or frequently changing information.

Fine-tuning means training a pre-trained model further on task-specific data. It changes the model's parameters and is useful when we want the model to learn a particular behavior, style, or specialized task.

So, in simple terms: **Prompting changes the instruction, RAG provides external knowledge, and fine-tuning changes the model's learned behavior.**”

□ Key Points to Memorize

Approach	What changes?	Best for
Prompting	Instructions	Simple task adaptation
RAG	External context	Private/current knowledge
Fine-tuning	Model parameters	Specialized behavior/task

◈ Small Example

Suppose you build a **company HR chatbot**.

Prompting 💬

“Answer professionally and briefly.”

👉 Changes **how the model responds**.

RAG 🗄

Retrieve:

Company HR policy PDF

Then give relevant information to the LLM.

👉 Changes **what information the model can use**.

Fine-tuning □

Train on thousands of company-specific examples.

👉 Changes **how the model behaves or performs a task**.

□ Easy Memory Trick

Prompting → Tell it 🗣️
RAG → Give it 📚
Fine-tuning → Train it □

⚠ ☆ Common Interview Question

Interviewer: "When would you choose RAG instead of fine-tuning?"

Good answer:

"I would prefer RAG when the information changes frequently or comes from private documents, because I can update the knowledge source without retraining the model. Fine-tuning is more appropriate when I need the model to learn a specific behavior, format, or task rather than simply access new information."

□ One-Line Memory

Prompt = Instructions
RAG = Knowledge
Fine-tuning = Behavior